CPSY 1291 · Recitation handout 5

PyTorch: tensors, autograd, and the training loop

The five lines that train every network in this course

Fall 2026 · Week 5 · hold this after Lecture 9; Assignment 2 is due Sun 10/18

What this session is for

Last week you derived a learning rule by hand and checked it numerically. This week you hand that job to a library.

You will not be asked to write a training loop from scratch in this course. You *will* be asked to read one, modify one, and diagnose one that is not working — and to know which of its lines are doing something and which are ceremony. That is the whole of this session.

> **The one idea.** PyTorch is NumPy that *remembers what you did to it*. Every operation on a tensor is recorded, so when you finally produce one number — the loss — the library can replay the recording backwards and hand you the derivative with respect to every parameter that took part. That replay is `loss.backward()`, and it is the only thing PyTorch does that NumPy cannot.

1 Tensors

A tensor is an array. Almost every NumPy habit transfers: indexing, slicing, `.shape`, broadcasting, `axis=` (spelled `dim=`).

```
import torch

x = torch.tensor([[1., 2.], [3., 4.]]) # from a Python list
x = torch.from_numpy(arr).float() # from a NumPy array -- note .float()
x.numpy() # and back again
x.shape # torch.Size([2, 2])
x.mean(dim=0) # 'axis' is spelled 'dim'
```

1.1 The dtype trap

NumPy defaults to `float64`. PyTorch defaults to `float32` and most layers refuse anything else. So converted arrays need an explicit cast:

```
X = torch.tensor(X_np, dtype=torch.float32) # do this
```

> **The error you will get, and what it really means.** `RuntimeError: expected scalar type Float but found Double`. "Double" is `float64`, i.e. a NumPy array that came across without a cast. It is never a bug in your model.

A second dtype rule: classification *targets* must be `long` (integers), not floats, and they are class indices — $0, 1, 2$ — not one-hot vectors.

1.2 Batch first, always

Every layer in PyTorch expects the first dimension to be the batch. A single example still needs that dimension:

```
x.shape # (784,) one image -- layers will reject this
x[None].shape # (1, 784) now it is a batch of one
```

The corresponding shapes for the data you will meet this term:

Data	Shape	Read as
tabular / flattened	(N, D)	N examples, D features
images	(N, C, H, W)	N images, C channels, height, width
sequences	(N, T, D)	N sequences, T steps, D features

Note that images are **channels-first** in PyTorch and channels-*last* in matplotlib. `plt.imshow` on a $(3, 64, 64)$ tensor fails; you need `img.permute(1, 2, 0)`.

2 Autograd in three lines

```
w = torch.tensor([2.0], requires_grad=True)
loss = (w ** 2).sum() # a graph is being recorded
loss.backward() # replay it backwards
w.grad # tensor([4.]) -- d(w^2)/dw = 2w = 4
```

Three facts that explain most autograd confusion:

1. **Gradients accumulate.** `.grad` is *added* to, not replaced, on every `backward()`. This is deliberate — it lets you sum gradients over several batches — but it means you must call `opt.zero_grad()` each step or you are descending on the sum of every gradient since the program started. Forget it and the loss typically diverges after a few epochs.
2. **`backward()` only works from a scalar.** The loss must be one number. If it is not, you forgot a `.mean()` or a `.sum()`.
3. **The graph is freed after the backward pass.** Calling `backward()` twice on the same loss raises an error. That error almost always means you built the loss outside the loop.

`torch.no_grad()`. Inside this block nothing is recorded. Use it whenever you are evaluating rather than training — computing validation accuracy, extracting features from a pretrained network, plotting. It is faster and it uses far less memory, and forgetting it is the usual reason a script runs out of memory during evaluation but not during training.

3 Building a model

3.1 `nn.Linear` is the neuron layer from lecture

`nn.Linear(in_features, out_features)` computes $\boldsymbol{x}\boldsymbol{W}^\top + \boldsymbol{b}$ — a whole layer of the units you studied last week, in one object. Its `weight` attribute has shape `(out, in)`, which is the transpose of what you might expect; that is a convention, and reading it wrong is a common source of confusion when you inspect a trained model.

```
import torch.nn as nn

model = nn.Sequential(
   nn.Linear(64, 16), # 64 inputs -> 16 hidden units
   nn.ReLU(),
   nn.Linear(16, 10), # 16 -> 10 class scores
)
sum(p.numel() for p in model.parameters()) # count the parameters
```

`nn.Sequential` is enough for everything in Assignment 2. Note that `ReLU` is a layer here, sitting *between* the linear layers — without it the whole stack collapses to a single linear map, which is exactly the point Lecture 9 makes about why depth needs nonlinearity.

3.2 The output layer, and the trap

The last layer emits **logits** — raw, unbounded scores, one per class. They are not probabilities and they are not meant to be.

Never put a ReLU or a softmax after your final layer if you are using `nn.CrossEntropyLoss`. That loss applies `log_softmax` internally. Applying softmax yourself first applies it twice and the model trains badly for reasons nothing will report. Applying *ReLU* first is worse: it clamps every negative logit to zero, so the model can no longer express "this class is unlikely" at all. Both are silent. Both are common in code found online.

Task	Loss	Final layer emits
regression	`nn.MSELoss()`	one number per target, no activation
classification	`nn.CrossEntropyLoss()`	raw logits, no activation
binary, if you must	`nn.BCEWithLogitsLoss()`	one raw logit

4 The training loop

Here it is, complete. Read it as five jobs, not as eleven lines.

```
model = nn.Sequential(nn.Linear(64, 16), nn.ReLU(), nn.Linear(16, 10))
lossfn = nn.CrossEntropyLoss()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(200):
   model.train()
   opt.zero_grad() # 1. forget last step's gradients
   out = model(Xtr) # 2. forward: predictions
   loss = lossfn(out, ytr) # 3. how wrong are we?
   loss.backward() # 4. gradient of loss w.r.t. every parameter
   opt.step() # 5. take one step downhill

   if epoch % 20 == 0:
      model.eval()
      with torch.no_grad():
         acc = (model(Xte).argmax(1) == yte).float().mean()
      print(f"{epoch:4d} loss {loss.item():.4f} test acc {acc:.3f}")
```

- `opt.zero_grad()` — because gradients accumulate.
- `loss.backward()` — the replay from the box on page 1.
- `opt.step()` — *this* is the line that changes the weights. Steps 1–4 only compute; nothing moves until here.
- `.item()` — pulls a Python float out of a one-element tensor. Printing the tensor itself works but drags the whole graph along.
- `model.train()` / `model.eval()` — switches the behavior of dropout and batch-norm layers. Harmless with the model above; a real bug the moment you use either, and free to write, so write it.

4.1 SGD or Adam?

	`SGD`	`Adam`
is	the update you derived last week	a per-parameter adaptive step
typical `lr`	10^{-2} to 10^{-1}	10^{-3}
use when	you want to see the rule work	you want it to just train

The learning rates are not interchangeable. Adam's default of 10^{-3} given to SGD trains imperceptibly slowly; SGD's 10^{-1} given to Adam usually diverges. When someone says a network "did not learn", the first question is which optimizer, and the second is what learning rate.

4.2 Reproducibility

```
torch.manual_seed(0) # weight init, shuffling, dropout
```

Same caveat as Recitation 3, and it is worth repeating in this new setting: seeding makes a run *reproducible*, not *meaningful*. A claim about an architecture needs several seeds, because weight initialization is random and the loss surface is not a bowl.

5 Using a pretrained network

You will spend more of this course *reading* networks than training them.

```
import torchvision
net = torchvision.models.alexnet(weights='DEFAULT')
net.eval() # not training: turn off train mode

W = net.features[0].weight.detach().numpy() # (64, 3, 11, 11)
```

`.detach()` removes the tensor from the autograd graph so it can become a NumPy array; without it you get an error telling you to call it. The shape reads as: 64 filters, each with 3 color channels, each 11×11 pixels. To display one, move the channel dimension last and rescale to $[0, 1]$:

```
f = W[10].transpose(1, 2, 0)
plt.imshow((f - f.min()) / np.ptp(f))
```

Pretrained vision models expect their inputs normalized the way they were trained — a specific per-channel mean and standard deviation. Skip that step and the filters still respond, but the numbers are not comparable to anything published. Whenever you feed a pretrained model your own images, find the normalization it was trained with and apply it.

6 When it does not learn

Work down this list in order. It is ordered by how often each one is the answer.

1. **Is the loss going down at all?** Print it every epoch, on a log axis. "Flat" and "slow" look identical on a linear axis.
2. **Did you call `opt.zero_grad()`?** Symptom: the loss falls for a few epochs, then explodes.
3. **Learning rate.** Try $\times 10$ and $\div 10$ before anything else. This fixes more cases than every other item combined.
4. **Are the targets the right dtype and shape?** Class indices as `long`, shape $(N,)$ — not one-hot, not $(N, 1)$.
5. **Is there a nonlinearity between the linear layers?** Without one the model is linear regardless of depth, and will plateau exactly where a linear model plateaus.

6. **Did you apply softmax or ReLU to the logits?** Silent, and it halves your accuracy.

7. **Can it overfit 10 examples?** Train on a tiny subset and try to drive the loss to zero. If it cannot, the bug is in the model or the loss. If it can, the model works and the problem is the data or the amount of training.

> **Item 7 is the one to remember.** "Can it memorize a handful of examples?" separates a broken pipeline from a hard problem in about thirty seconds, and it is the first thing an experienced person does. A model that cannot overfit ten points will never generalize from ten thousand.

7 Exercises

1. You write `model(X)` and get `expected scalar type Float but found Double`. What did you do, and what is the fix?

2. Your loss decreases for 5 epochs, then grows without bound and reaches `nan`. Give the two most likely causes and how to tell them apart.

3. A classmate's classifier ends with `nn.Linear(16, 10)` followed by `nn.ReLU()`, trained with `nn.CrossEntropyLoss`. It reaches 60% where yours reaches 95%. Explain what the ReLU did.

4. `nn.Sequential(nn.Linear(64,16), nn.Linear(16,10))`. How many parameters, and what function class can this model express?

5. You extract features from a pretrained network inside a `for` loop over 5,000 images and run out of memory. One line fixes it. Which?

6. Your model gets 100% training accuracy on 10 examples and 12% test accuracy on 10,000. Is the pipeline broken?

Answers

1. You passed a NumPy `float64` array (or a tensor built from one) into a layer expecting `float32`. Fix: `torch.tensor(X, dtype=torch.float32)`.

2. (i) A missing `opt.zero_grad()`, so gradients accumulate and each step grows; (ii) a learning rate too large. Tell them apart by fixing `zero_grad` first — if it still diverges, the learning rate is the cause. A missing `zero_grad` typically shows a few good epochs first; too large a learning rate usually misbehaves from the start.

3. `CrossEntropyLoss` expects raw logits. ReLU clamps every negative logit to zero, so the model can no longer express "this class is unlikely" — all the evidence *against* a class is destroyed, leaving only evidence for. Nothing errors.

4. $(64{\times}16 + 16) + (16{\times}10 + 10) = 1{,}040 + 170 = 1{,}210$ parameters. With no nonlinearity between them it can express exactly the linear maps from $\mathbb{R}^{64}$ to $\mathbb{R}^{10}$ — the same class as a single `nn.Linear(64, 10)`, with more parameters and no more power.

5. Wrap the loop in `with torch.no_grad():`. Without it, PyTorch keeps the graph for all 5,000 forward passes.

6. No — that is the expected outcome of the overfit-ten-examples check, and it tells you the model and loss are wired correctly. It says nothing about generalization, which is a separate question and the subject of next week.

Cheat sheet

You want	You write
NumPy → tensor	`torch.tensor(a, dtype=torch.float32)`
tensor → NumPy	`t.detach().numpy()`
add a batch dimension	`x[None]`
image for `imshow`	`img.permute(1, 2, 0)`
a small MLP	`nn.Sequential(nn.Linear(..), nn.ReLU(), ..)`
count parameters	`sum(p.numel() for p in m.parameters())`
one training step	`zero_grad; forward; loss; backward; step`
evaluate without recording	`with torch.no_grad():`
predicted class	`out.argmax(1)`
a Python float from a tensor	`loss.item()`
a pretrained network	`torchvision.models.alexnet(weights='DEFAULT')`
same run twice	`torch.manual_seed(0)`